

Modified Entropy Maximizing for Wordle Solvers: An Analysis of the Trade-off between Partition Diversity and Uniformity

Useong Jeong

Abstract

This report interprets the Wordle solving problem as a problem of partitioning the candidate answer set according to feedback patterns, and proposes a Modified Entropy Maximizing (MEM) method that controls the trade-off between partition diversity and partition uniformity. The standard entropy score measures the amount of information in the feedback distribution generated by a guess word, but it does not clearly distinguish whether that value comes from the ability to generate many feedback blocks or from the ability to divide block sizes uniformly. To address this issue, this study defines a diversity score $B(g)$, which reflects the number of non-empty feedback blocks, and a uniformity score $U(g)$, which reflects the uniformity of block sizes. It then proposes the MEM score $M_\lambda(g)$, which controls the relative weight of these two components. In particular, when $\lambda = 0.5$, the proposed score takes the same form as normalized standard entropy, and changes in λ allow comparison between diversity-oriented and uniformity-oriented strategies.

In the experiments, the performance of the solver was compared for $\lambda = 0.0$ through $\lambda = 1.0$ using 2,309 Wordle answer candidates and 12,947 allowed guess words. The results showed that the average number of attempts for the standard entropy baseline, $\lambda = 0.5$, was 3.4638, while settings that placed greater emphasis on partition diversity generally produced lower average numbers of attempts. In particular, $\lambda = 1.0$ achieved the best average-case performance, with an average of 3.4305 attempts. On the other hand, $\lambda = 0.7$ had a slightly higher average number of attempts, 3.4353, but solved all answers within five attempts and achieved the highest proportion of solutions within four attempts, showing a more stable solution distribution. These results demonstrate that explicitly controlling the trade-off between diversity and uniformity, which together constitute entropy in a Wordle solver, is useful for analyzing both average performance and worst-case stability.

1 Introduction

Wordle is a word-guessing game in which the player attempts to identify a hidden five-letter word within a limited number of tries. At each attempt, the user enters one word and receives feedback for each letter in the form of green, yellow, or gray. Green indicates that the letter is in the correct position in the answer, while yellow indicates that the letter is included in the answer but is in a different position. Gray indicates that the letter is not included in the answer, or that the number of occurrences of that letter has already been accounted for by feedback in other positions. Although these rules appear simple, Wordle can essentially be interpreted as an information search problem in which the set of possible answers is reduced using limited feedback.

An important problem in Wordle solving algorithms is deciding which word to guess at each step. Let the current set of possible answer candidates be C , and let a guess word be g . Each candidate answer $a \in C$ produces one feedback pattern $p(a, g)$ for the guess word g . Therefore, the guess word g serves to divide the current candidate set C into subsets according to feedback patterns. This

can be represented as follows:

$$C_p(g) = \{a \in C : p(a, g) = p\}.$$

In other words, a single guess partitions the current possible answer candidates into multiple feedback blocks. A good guess can be understood as one that makes the remaining candidate set as small as possible after the actual feedback is received.

From this perspective, one widely used method is entropy-based guess selection. Let the sizes of the non-empty partition blocks generated by a guess word g be $s_1, s_2, \dots, s_k$. Here, k is the number of non-empty feedback blocks, and $n = |C|$ is the number of current answer candidates. The probability of each feedback block occurring can be defined as follows:

$$q_i = \frac{s_i}{n}.$$

The entropy of the guess word g is then given by:

$$H(g) = - \sum_{i=1}^k q_i \log_2 q_i.$$

A high entropy value can be interpreted as meaning that the guess word generates diverse feedback outcomes and distributes the candidates relatively well. Thus, a standard entropy-based solver selects the guess word with the largest $H(g)$ at each step.

However, the entropy score combines several properties of a partition into a single value. For example, one guess may generate a very large number of feedback blocks, but most candidates may be concentrated in one of those blocks. Conversely, another guess may generate fewer blocks, but divide the candidates relatively uniformly. Both cases can be evaluated using entropy, but entropy does not clearly reveal whether the guess is good because it “creates many blocks” or because it “makes the block sizes uniform.”

This study focuses on this point and separates the quality of a partition in Wordle guess selection into two components. The first component is partition diversity, which represents how many non-empty feedback blocks a guess word generates. The second component is partition uniformity, which represents how uniformly the candidate answers are distributed among the generated blocks. This study proposes an MEM score function that combines these two components and uses a control variable $\lambda \in [0, 1]$ to adjust their relative weights.

In the proposed scoring function, the case $\lambda = 0.5$ can be interpreted as equivalent to the conventional normalized entropy score. When $\lambda > 0.5$, the solver relatively favors guess words that generate more feedback blocks, whereas when $\lambda < 0.5$, it places greater importance on the uniformity of the generated blocks. Thus, the proposed method generalizes the existing entropy-based approach while enabling explicit control of the trade-off between partition diversity and uniformity in the Wordle solving process.

The purpose of this report is to experimentally analyze how the MEM score function affects Wordle solving performance. To this end, the solver is run for multiple λ values using a Wordle answer candidate list and an allowed guess word list, and the average number of attempts, maximum number of attempts, first guess word, and distribution of attempt counts are compared. In particular, using $\lambda = 0.5$, which corresponds to standard entropy, as the baseline, this study examines how performance changes when greater weight is placed on partition diversity or uniformity.

The remainder of this report is organized as follows. Section 2 explains Wordle feedback partitions and the proposed MEM score function. Section 3 presents the dataset, experimental settings, and evaluation metrics used in the experiments. Section 4 analyzes the experimental results according to the value of λ . Finally, Section 5 summarizes the study and presents directions for future work.

2 Method

This section formalizes the Wordle solving process as a feedback-based partitioning problem and explains the MEM score function proposed in this study. The key idea of the proposed method is not to evaluate the partition generated by a single guess solely through entropy, but to evaluate partition diversity and partition uniformity separately.

2.1 Wordle Feedback Partitions

In Wordle, a single guess word divides the current set of possible answer candidates into multiple subsets according to feedback patterns. Let the current set of possible answer candidates be C , and let the set of allowed guess words be G . For a guess word $g \in G$, each candidate answer $a \in C$ generates one feedback pattern $p(a, g)$.

In this study, Wordle feedback is encoded as follows:

$$0 = \text{gray}, \quad 1 = \text{yellow}, \quad 2 = \text{green}.$$

Because Wordle uses five-letter words, the maximum number of possible feedback patterns is

$$3^5 = 243.$$

Given a guess word g , the candidate answers that generate the same feedback pattern form a single block. The block corresponding to a feedback pattern p is defined as follows:

$$C_p(g) = \{a \in C : p(a, g) = p\}.$$

The set of non-empty $C_p(g)$ forms a partition of the current candidate set C .

A good guess word should divide the candidate set in a way that provides a high amount of information. That is, a guess word that partitions the current candidates into many small blocks is advantageous, since it reduces the size of the remaining candidate set after the actual feedback is received. Therefore, the Wordle solving problem can be viewed as the problem of selecting, at each step, the guess word that most effectively partitions the current candidate set.

2.2 MEM Score

A conventional entropy-based Wordle solver calculates the amount of information in the feedback partition generated by each guess word. Let the sizes of the non-empty partition blocks generated by a guess word g be

$$s_1, s_2, \dots, s_k.$$

Here, k is the number of non-empty feedback blocks, and the number of current candidates is

$$n = |C|.$$

The probability of each block is defined as follows:

$$q_i = \frac{s_i}{n}.$$

Then, the entropy of the guess word g is given by:

$$H(g) = - \sum_{i=1}^k q_i \log_2 q_i.$$

The larger the entropy, the more informatively the guess word can be considered to divide the candidate set. However, entropy combines two properties of a partition into a single value. The first is how many feedback blocks are generated, and the second is how uniform the sizes of the generated blocks are. In this study, these two components are separated into partition diversity and partition uniformity.

First, the partition diversity score $B(g)$ is defined as follows:

$$B(g) = \frac{\log_2 k}{\log_2(\min(243, n))}.$$

The denominator $\min(243, n)$ represents the maximum possible number of non-empty blocks in the current state. Although there can be at most 243 feedback patterns, if there are n candidate answers, the number of non-empty blocks cannot exceed n . Thus, $B(g)$ represents how many diverse feedback outcomes the guess word g generates relative to the maximum possible number of blocks in the current state.

Next, the partition uniformity score $U(g)$ is defined as follows:

$$U(g) = \frac{H(g)}{\log_2 k}.$$

Here, $\log_2 k$ is the maximum possible entropy when the k blocks are perfectly uniform. Thus, $U(g)$ indicates how uniformly the candidate answers are distributed among the generated blocks. The closer $U(g)$ is to 1, the more uniform the block sizes are, while the closer it is to 0, the more concentrated the candidates are in a particular block.

The MEM score proposed in this study is defined by combining $B(g)$ and $U(g)$:

$$M_\lambda(g) = B(g)^{2\lambda} \cdot U(g)^{2(1-\lambda)}.$$

Here, $\lambda \in [0, 1]$ is a parameter that controls the relative weight between partition diversity and partition uniformity.

When $\lambda > 0.5$, the influence of $B(g)$ becomes larger, so guess words that generate more feedback blocks are relatively preferred. Conversely, when $\lambda < 0.5$, the influence of $U(g)$ becomes larger, so guess words that make the sizes of the generated blocks more uniform are preferred.

In particular, when $\lambda = 0.5$, the proposed score becomes equivalent in form to the conventional normalized entropy score:

$$M_{0.5}(g) = B(g) \cdot U(g).$$

Expanding this gives:

$$M_{0.5}(g) = \frac{\log_2 k}{\log_2(\min(243, n))} \cdot \frac{H(g)}{\log_2 k}.$$

Therefore,

$$M_{0.5}(g) = \frac{H(g)}{\log_2(\min(243, n))}.$$

In other words, $\lambda = 0.5$ corresponds to the standard entropy score normalized by the maximum possible entropy in the current candidate set.

2.3 Solver Procedure

A solver using the proposed scoring function evaluates possible guess words for the current candidate set C at each step and selects the word with the highest MEM score. The overall procedure is as follows.

1. Set the current possible answer candidate set C .
2. For each allowed guess word $g \in G$, compute the feedback partition.
3. From each partition, compute the entropy $H(g)$, diversity score $B(g)$, and uniformity score $U(g)$.
4. Select the guess word with the highest MEM score $M_\lambda(g)$.
5. Reduce the candidate set to the corresponding partition block based on the feedback pattern for the selected guess word.
6. Repeat the above process until the answer is found.

In the basic experiments of this study, the entire allowed guess word set G is used as the evaluation target at every step. This means that words not included in the current candidate answer set C may also be used as guess words for information acquisition. For example, even if a word is not an actual answer candidate, it can be selected as the next guess if it effectively partitions the remaining candidates.

In the implementation, a precomputed pattern table for all pairs of answer candidates and allowed guess words is used to improve computational efficiency. The pattern table is defined as follows:

$$T[i, j] = p(a_i, g_j),$$

where a_i is the i -th answer candidate and g_j is the j -th allowed guess word. Using this table, the feedback distribution for the current candidate set can be quickly retrieved at each step without repeatedly recalculating feedback patterns.

In addition, because the same candidate set may appear repeatedly across multiple solving paths, the solver can reuse results for states that have already been computed. This approach helps reduce computational cost in experiments where the solver is repeatedly run for multiple λ values.

3 Experiments

This section explains the experimental setup used to evaluate the performance of the proposed MEM-based Wordle solver. The purpose of the experiments is to analyze how the control parameter λ in the MEM score function affects first-guess selection and overall solving performance. To this end, the same answer candidate set and allowed guess word set were used, and the solver was repeatedly run for various λ values.

3.1 Dataset

The experiments used a list of five-letter Wordle words. The word list is divided into a set of words that can be actual answers and a set of allowed words that users can enter as guesses. In this report, the answer candidate set is denoted by A , and the allowed guess word set is denoted by G .

The number of answer candidates used in the experiments is:

$$|A| = 2309.$$

The allowed guess word set was constructed to include the answer candidates, and the total number of allowed guess words is:

$$|G| = 12947.$$

All answer candidates are also included in the allowed guess word set, so

$$A \subseteq G$$

holds.

In each experiment, every word included in the answer candidate set A was used once as the actual answer. That is, the solver solved a total of 2,309 Wordle problems, and the number of guesses required for each answer was recorded. This method makes it possible to evaluate not only performance for a specific answer, but also the average solver performance over the entire answer candidate set.

3.2 Evaluation Metrics

The solver’s performance was evaluated based on the number of guesses required to find each answer. Let $t(a)$ be the number of guesses required for the solver to find the answer $a \in A$. The most important evaluation metric is the average number of attempts:

$$\text{Mean Attempts} = \frac{1}{|A|} \sum_{a \in A} t(a).$$

A smaller value means that the solver finds the answer with fewer attempts on average.

The maximum number of attempts was also measured to examine worst-case performance:

$$\text{Max Attempts} = \max_{a \in A} t(a).$$

Because Wordle is a game in which the answer must be found within a limited number of attempts, it is important to evaluate not only average performance but also whether certain answers require an excessively large number of guesses.

In this experiment, the following items were recorded for each λ value:

- Selected first guess word
- Average number of attempts
- Maximum number of attempts
- Distribution of the number of answers by attempt count
- List of answers that produced the maximum number of attempts

The first guess word is the word with the highest MEM score for the entire initial candidate set under the corresponding λ value. Therefore, when the value of λ changes, the first guess word may also change. This makes it possible to examine whether λ affects not only the subsequent search process but also the initial information search strategy.

3.3 Lambda Sweep

In this experiment, solver performance was compared by varying the value of λ in the MEM score function

$$M_\lambda(g) = B(g)^{2\lambda} \cdot U(g)^{2(1-\lambda)}.$$

The values of λ were set as follows:

$$\lambda \in \{0.0, 0.1, 0.2, \dots, 1.0\}.$$

For each λ value, the solver did not fix the first guess word in advance. Instead, it selected as the first guess the word with the highest $M_\lambda(g)$ value based on the initial candidate set A and the entire allowed guess word set G . After that, the candidate set was reduced according to the feedback, and the same scoring function was used to select the next guess at each step.

In the basic setting of this experiment, the entire allowed guess word set G was used as the evaluation target at every step. That is, even if a word was not included in the current possible answer candidate set, it could still be selected as a guess for information acquisition if it effectively partitioned the candidates. Although this setting increases computational cost, it better reflects the information search ability of an entropy-based solver.

The case $\lambda = 0.5$ is used as an important comparison baseline. As explained earlier, when $\lambda = 0.5$, the proposed MEM score takes the same form as the normalized standard entropy score. Therefore, in this experiment, $\lambda = 0.5$ is regarded as the baseline standard entropy-based solver.

In contrast, when $\lambda < 0.5$, greater weight is placed on partition uniformity $U(g)$, while when $\lambda > 0.5$, greater weight is placed on partition diversity $B(g)$. Thus, the λ sweep makes it possible to analyze how the solver’s average performance and worst-case performance change when uniformity or diversity is emphasized more strongly than in the standard entropy criterion.

4 Results and Discussion

This section analyzes changes in solver performance according to the value of λ . The experiments were conducted from $\lambda = 0.0$ to $\lambda = 1.0$ in increments of 0.1. For each setting, the selected first guess word, average number of attempts, maximum number of attempts, and the number of answers that produced the maximum number of attempts were recorded.

4.1 Overall Performance Changes According to λ

Table 1 shows the overall performance of the solver according to the value of λ .

Table 1: Comparison of Wordle solver performance according to λ

λ	First guess	Mean attempts	Max attempts	Worst count
0.0	roate	5.0645	9	4
0.1	roate	3.5219	5	47
0.2	roate	3.4937	5	44
0.3	roate	3.4799	5	42
0.4	roate	3.4773	5	44
0.5	soare	3.4638	6	1
0.6	reast	3.4361	5	53
0.7	reast	3.4353	5	54
0.8	trace	3.4309	6	2
0.9	trace	3.4318	6	2
1.0	trace	3.4305	6	1

The first observation from Table 1 is that the value of λ directly affects first-guess selection. From $\lambda = 0.0$ to $\lambda = 0.4$, **roate** was selected as the first guess. At $\lambda = 0.5$, which corresponds to standard normalized entropy, **soare** was selected. Then, at $\lambda = 0.6$ and $\lambda = 0.7$, **reast** was selected, while for $\lambda \geq 0.8$, **trace** was selected as the first guess. This shows that as λ adjusts the

relative weight between partition uniformity and partition diversity, the word judged to partition the initial candidate set most effectively also changes.

In terms of the average number of attempts, $\lambda = 1.0$ recorded the lowest average number of attempts, 3.4305. This value is lower than the average number of attempts, 3.4638, for the standard entropy baseline $\lambda = 0.5$. In addition, $\lambda = 0.6$, $\lambda = 0.7$, $\lambda = 0.8$, and $\lambda = 0.9$ all showed lower average numbers of attempts than $\lambda = 0.5$. Therefore, under the experimental setting of this study, settings that place greater weight on partition diversity than standard entropy tended to improve average solving performance.

In contrast, when $\lambda = 0.0$, the average number of attempts increased significantly to 5.0645, and the maximum number of attempts increased to 9. The setting $\lambda = 0.0$ is an extreme case in which the MEM score emphasizes only partition uniformity. In this case, the number of generated feedback blocks is not sufficiently considered, so even if the blocks are uniform, the entire candidate set may not be divided finely enough. In other words, in a Wordle solver, it is important not only to create a uniform partition but also to generate as many meaningful feedback blocks as possible.

Looking at the maximum number of attempts, the maximum was 5 attempts for $\lambda = 0.1$ through $\lambda = 0.4$, as well as for $\lambda = 0.6$ and $\lambda = 0.7$. In contrast, the maximum was 6 attempts for $\lambda = 0.5$, $\lambda = 0.8$, $\lambda = 0.9$, and $\lambda = 1.0$. Therefore, if only the average number of attempts is considered, $\lambda = 1.0$ is the best setting. However, when worst-case performance is also considered, $\lambda = 0.6$ or $\lambda = 0.7$ can be seen as a more balanced setting. In particular, $\lambda = 0.7$ achieves both an average number of attempts of 3.4353 and a maximum of 5 attempts, showing a good trade-off between average performance and worst-case performance.

Here, Worst count refers to the number of answers that reached the maximum number of attempts in each setting. Therefore, when comparing different λ values, Worst count should be interpreted together with Max attempts rather than on its own. For example, the Worst count for $\lambda = 0.5$ is 1, but its maximum number of attempts is 6, whereas the Worst count for $\lambda = 0.7$ is 54, but all of these cases were solved within 5 attempts. Considering Wordle’s attempt limit, $\lambda = 0.7$ can be regarded as a more stable setting in that it maintains the maximum number of attempts at 5.

4.2 Attempt Count Distribution and Stability Analysis

Next, this study compared the attempt count distributions of $\lambda = 0.5$, the standard entropy baseline; $\lambda = 0.7$, a balanced setting; and $\lambda = 1.0$, which achieved the lowest average number of attempts. The results are shown in Table 2.

Table 2: Comparison of attempt count distributions for $\lambda = 0.5$, $\lambda = 0.7$, and $\lambda = 1.0$

Attempts	$\lambda = 0.5$	$\lambda = 0.7$	$\lambda = 1.0$
1	0 (0.00%)	0 (0.00%)	1 (0.04%)
2	45 (1.95%)	69 (2.99%)	74 (3.20%)
3	1213 (52.53%)	1220 (52.84%)	1232 (53.36%)
4	987 (42.75%)	966 (41.84%)	935 (40.49%)
5	63 (2.73%)	54 (2.34%)	66 (2.86%)
6	1 (0.04%)	0 (0.00%)	1 (0.04%)

As shown in Table 2, $\lambda = 0.7$ solved the largest number of answers within four attempts among the three settings. Out of the 2,309 total answers, $\lambda = 0.7$ solved 2,255 answers within four attempts, corresponding to 97.66% of the total. In contrast, the standard entropy baseline $\lambda = 0.5$ solved 2,245 answers within four attempts, while $\lambda = 1.0$, which had the lowest average number of attempts,

solved 2,242 answers within four attempts. Therefore, based on the proportion of answers solved within four attempts, $\lambda = 0.7$ can be said to have shown the most stable performance.

This result reveals a difference that cannot be captured by the average number of attempts alone. Because $\lambda = 1.0$ solves more answers within 1, 2, and 3 attempts, it achieves the lowest average number of attempts. However, it requires 5 or more attempts for 67 answers, which is more than the 54 answers for $\lambda = 0.7$. In other words, $\lambda = 1.0$ has the advantage of solving more answers very quickly, but it may produce longer solution paths for some answers.

In contrast, $\lambda = 0.7$ increases the number of 2-attempt and 3-attempt solutions compared to the baseline while minimizing cases that require 5 or more attempts. In particular, under $\lambda = 0.7$, no answer required 6 attempts, and the number of answers requiring 5 or more attempts was 54, the smallest among the three settings. Considering that Wordle is a game with a maximum limit of 6 attempts, having many answers solved within 4 attempts provides a different kind of stability from simply having a low average. That is, $\lambda = 0.7$ tends to solve most answers well before reaching the attempt limit.

In terms of the total number of attempts, $\lambda = 0.5$ required 7,998 guesses to solve all 2,309 answers. In contrast, $\lambda = 0.7$ required 7,932 total guesses, and $\lambda = 1.0$ required 7,921 total guesses. Thus, $\lambda = 0.7$ reduced the total number of guesses by 66 compared with the standard entropy baseline, while $\lambda = 1.0$ reduced it by 77. This shows that settings that more strongly reflect partition diversity improved average solving efficiency.

Taken together, these results show that λ is an important parameter that determines the character of the solver. If λ is too small, the solver may overemphasize partition uniformity and fail to obtain sufficient candidate elimination. As λ increases, the solver increasingly favors guess words that generate more feedback blocks, and in this experiment, this direction was effective in reducing the average number of attempts. However, the setting with the best average performance was not always the best in terms of maximum number of attempts or stability of the solving distribution. Although $\lambda = 1.0$ showed the best average-case performance with an average of 3.4305 attempts, its maximum number of attempts was 6, and it required 5 or more attempts for more answers than $\lambda = 0.7$. By contrast, $\lambda = 0.7$ had a slightly higher average number of attempts than $\lambda = 1.0$, at 3.4353, but solved the largest number of answers within four attempts and solved all answers within five attempts. Therefore, $\lambda = 1.0$ can be interpreted as the setting that minimizes the average number of attempts, while $\lambda = 0.7$ can be interpreted as a balanced setting when considering both the proportion of answers solved within four attempts and worst-case stability.

5 Conclusion

This report interpreted the Wordle solving problem as a problem of partitioning the candidate set according to feedback patterns, and extended the existing entropy-based guess selection method from the perspective of partition diversity and partition uniformity. To this end, this study defined a diversity score $B(g)$, which represents the number of non-empty feedback blocks generated by a guess word, and a uniformity score $U(g)$, which represents how uniformly the sizes of the generated blocks are distributed. It also proposed the MEM score $M_\lambda(g)$, which includes a parameter λ that controls the relative importance of these two components. This scoring function takes the same form as normalized standard entropy when $\lambda = 0.5$, and by adjusting λ , the conventional entropy-based solver can be interpreted in a more general form.

The experimental results confirmed that the value of λ is an important parameter that affects both first-guess selection and overall solving performance. Under $\lambda = 0.5$, which corresponds to the standard entropy baseline, the average number of attempts was 3.4638. However, in the range

$\lambda > 0.5$, where greater weight is placed on partition diversity, the average number of attempts was generally lower. In particular, $\lambda = 1.0$ showed the best average-case performance, with an average of 3.4305 attempts. This indicates that, under the experimental setting of this study, a strategy that divides the candidate set into as many feedback blocks as possible is effective in improving average solving efficiency.

However, it is difficult to fully evaluate solver performance using only the average number of attempts. Although $\lambda = 1.0$ recorded the lowest average number of attempts, its maximum number of attempts was 6, and relatively long solution paths occurred for some answers. By contrast, $\lambda = 0.7$ had a slightly higher average number of attempts than $\lambda = 1.0$, at 3.4353, but solved all answers within five attempts and achieved the highest proportion of answers solved within four attempts. Therefore, if average performance is the top priority, $\lambda = 1.0$ is advantageous, whereas if worst-case stability and the solving distribution are also considered, $\lambda = 0.7$ can be interpreted as the more balanced choice.

These results show that, in a Wordle solver, it is meaningful to analyze the components that constitute entropy separately rather than simply maximizing the entropy value. In particular, when $\lambda = 0.0$, which excessively emphasizes only partition uniformity, both the average and maximum numbers of attempts deteriorated significantly. This suggests that creating a uniform partition alone is not sufficient, and that the ability to generate enough feedback blocks to finely separate the candidate set is important. As a result, the proposed MEM score provides a method for explicitly controlling the trade-off between diversity and uniformity in the Wordle guess selection process.

Future research can extend this study in several directions. First, this report applied a fixed λ value throughout the entire solving process, but methods that dynamically adjust λ according to the size of the candidate set or the solving stage could be considered. For example, partition diversity could be emphasized more strongly in the early stage, while uniformity or a strategy prioritizing answer candidates could be reflected more strongly in the later stage when the number of candidates has decreased. Second, evaluation from a multi-objective optimization perspective is also possible, considering not only the average number of attempts but also the failure probability, the proportion of answers solved within four attempts, and computation time. Third, combining the MEM score with tie-breaking rules or strategies that prioritize words within the candidate set could further improve its usefulness in practical Wordle-playing environments.

In summary, this study is significant in that it decomposes the entropy-based approach to Wordle solving into two components, partition diversity and partition uniformity, and generalizes it into a controllable scoring function. Experimentally, settings that emphasize diversity more than standard entropy were able to improve average performance, and in particular, $\lambda = 0.7$ showed a good balance between average performance and worst-case stability. Therefore, the proposed MEM score can be used as a useful criterion for analyzing and adjusting the performance of Wordle solvers.